1 Vulnerability Management Procedures

Fuzzy Monster - QuizTimer4Zoom Application Effective Date: 2025-10-28 **Last Updated:** 2025-10-28 **Company:** Fuzzy Monster **Domain:** fuzzy.monster **Contact:** security@fuzzy.monster

---

1.1 1. Introduction

This document outlines the comprehensive vulnerability management procedures for QuizTimer4Zoom. The vulnerability management process is designed to identify, assess, prioritize, remediate, and monitor security vulnerabilities throughout the application lifecycle.

1.1.1 1.1 Purpose

To establish a systematic approach for: - Discovering vulnerabilities before they impact users - Assessing and prioritizing remediation efforts - Implementing timely fixes with verification - Monitoring for new vulnerabilities on an ongoing basis - Maintaining transparency with stakeholders

1.1.2 1.2 Scope

This policy applies to: - All QuizTimer4Zoom application code and dependencies - Third-party libraries and packages (npm dependencies) - Infrastructure and deployment configurations - Development, testing, staging, and production environments

1.1.3 1.3 Roles & Responsibilities

Security Lead: security@fuzzy.monster - Overall vulnerability management strategy - Risk assessment and prioritization - Communication with Zoom and stakeholders - Response time SLA enforcement

Development Team: dev@fuzzy.monster - Code security reviews - Vulnerability remediation - Testing and verification - Deployment coordination

QA/Testing Team: - Vulnerability validation - Test case execution - Regression testing after fixes - Compliance verification

---

1.2 2. Vulnerability Discovery

1.2.1 2.1 Automated Scanning

Dependency Vulnerability Scanning:

Tool	Frequency	Purpose	Command
npm audit	Weekly	Dependency vulnerabilities	`npm audit`
ESLint + Security Plugin	Every commit	Code pattern issues	`npm run lint`
Semgrep	Weekly	SAST scanning	`semgrep --config=p/security-audit`

Process: 1. Run automated scans in CI/CD pipeline 2. Generate reports and store in logs 3. Review findings and categorize 4. Alert Security Lead if issues found 5. Track all vulnerabilities in vulnerability database

1.2.2 2.2 Manual Code Review

Security-Focused Code Review: - All pull requests reviewed for security issues - Focus on: authentication, authorization, data handling, error handling - Checklist-based review process - Peer review by at least one team member

Areas of Review: - Input validation and sanitization - Authentication and session management - Authorization and access control - Cryptographic implementations - Data protection and privacy - Error handling and logging - Dependency usage and configuration

1.2.3 2.3 Third-Party Vulnerability Advisories

Monitoring Sources: - GitHub Security Advisories (GHSA) - National Vulnerability Database (NVD) - CVE.org - Zoom Security Bulletins - npm Security Advisories - ESLint security plugin updates

Process: 1. Subscribe to relevant advisory feeds 2. Monitor Zoom security announcements 3. Check for advisories affecting current dependencies 4. Act on critical advisories within 24 hours

1.2.4 2.4 Vulnerability Disclosure Policy

Responsible Disclosure: - Security researchers can report issues to security@fuzzy.monster - Expected response time: 24-48 hours - Fix timeline: Based on severity (see section 3) - Disclosure: After fix is deployed (with researcher agreement)

1.3 3. Vulnerability Assessment & Prioritization

1.3.1 3.1 Severity Classification

All vulnerabilities are classified using **CVSS 3.1** scoring:

Severity	CVSS Score	Response Time	Fix Timeline
Critical	9.0 - 10.0	1 hour	24 hours
High	7.0 - 8.9	4 hours	1 week
Medium	4.0 - 6.9	1 business day	2 weeks
Low	0.1 - 3.9	2 business days	1 month

1.3.2 3.2 Risk Assessment

Each vulnerability is assessed for:

Exploitability: - Ease of exploitation (low/medium/high) - Attack vector (network/adjacent/local) - Attack complexity (low/high) - Privileges required (none/low/high) - User interaction needed (yes/no)

Impact: - Confidentiality impact (none/low/high) - Integrity impact (none/low/high) - Availability impact (none/low/high) - Scope (unchanged/changed)

Environmental Factors: - Whether it affects application runtime or build tools only - Whether it’s in transitive vs. direct dependencies - Number of affected users - Business criticality of affected feature

1.3.3 3.3 Prioritization Matrix

```
Priority = CVSS_Score × Exploitability × Impact × Urgency

Priority 1 (Critical): Fix immediately
- CVSS >= 9.0, OR
- Actively exploited, OR
- Affects user data/authentication

Priority 2 (High): Fix within 1 week
- CVSS 7.0-8.9, OR
- High exploitability + high impact
- Affects application security controls

Priority 3 (Medium): Fix within 2 weeks
- CVSS 4.0-6.9
- Moderate exploitability or impact
- Affects specific features

Priority 4 (Low): Fix within 1 month
- CVSS < 4.0
- Low exploitability and impact
- Does not affect core functionality
```

1.3.4 3.4 Assessment Documentation

Each vulnerability receives: - CVSS score with justification - CWE classification - Affected component identification - Runtime vs. build-time impact analysis - Exploitability assessment - Business risk rating - Remediation recommendations

---

1.4 4. Remediation Procedures

1.4.1 4.1 Remediation Planning

Step 1: Develop Fix Strategy - Determine fix approach (update, patch, mitigation, accept) - Identify affected components - Plan testing approach - Estimate effort and timeline

Step 2: Development - Implement fix in feature branch - Code review required before merge - Document changes in commit message

Step 3: Testing - Unit tests for the fix - Integration tests for affected components - Regression tests (full test suite) - Security testing specific to vulnerability

Step 4: Staging Verification - Deploy to staging environment - Run full test suite - Verify vulnerability is fixed - Check for unintended side effects

Step 5: Production Deployment - Create deployment plan - Schedule deployment window - Execute deployment - Monitor application health

Step 6: Verification & Monitoring - Re-run vulnerability scans - Monitor logs for errors - Check user reports - Document resolution

1.4.2 4.2 Remediation Options

1.4.2.1 Option A: Update Dependency

```
npm update [package]
npm audit fix
```

- Most common approach
- Automatic with npm
- Verify compatibility first

1.4.2.2 Option B: Patch Version

```
npm install package@[specific-version]
```

- Install specific version
- Useful if latest has breaking changes
- Still need testing

1.4.2.3 Option C: Mitigation

- Code-level fixes when dependency update not available
- Configuration changes
- Input validation additions
- Output encoding fixes

1.4.2.4 Option D: Accept Risk

- Document in vulnerability manifest
- Only for low-risk transitive dependencies
- Risk acceptance form required
- Regular re-assessment

1.4.3 4.3 Remediation Verification

After Fix Applied:

1. **Automated Verification**
   - Run `npm audit` (should show fixed or reduced count)
   - Run ESLint security scan
   - Run Semgrep SAST scan
2. **Manual Verification**
   - Code review of fix
   - Test case execution
   - Functionality testing
   - Security testing
3. **Documentation**
   - Record fix in vulnerability database
   - Document verification results
   - Update SECURITY_SCAN_REPORT.md

1.5 5. Transitive Dependency Management

1.5.1 5.1 Understanding Transitive Dependencies

Transitive dependencies are packages installed as dependencies of your direct dependencies.

Example:

```
quiztimer4zoom
└── vercel@48.6.0
    ├── @vercel/node
    │   └── path-to-regexp  ← Transitive (vulnerability here)
    └── @vercel/express
        └── esbuild         ← Transitive (vulnerability here)
```

1.5.2 5.2 Transitive Vulnerability Assessment

Key Questions:

1. **Is it exploitable at runtime?**
   - Does it execute in the running application?
   - Or only during build/deployment?
   - Answer determines impact level
2. **Can it be safely fixed?**
   - Would fixing introduce new vulnerabilities?
   - Would it require downgrading other packages?
   - Would it break application functionality?
3. **Is the fix worth the cost?**
   - How much effort to remediate?
   - Are there unintended consequences?
   - What is the actual risk vs. effort ratio?

1.5.3 5.3 Transitive Vulnerability Decision Framework

Scenario	Action	Rationale
Affects runtime + fixable	FIX	Security impact + feasible
Affects runtime + not fixable	MITIGATE	Can’t fix but need to reduce risk
Build-time only + fixable	OPTIONAL	Lower priority, fix if low effort
Build-time only + not fixable	ACCEPT	No impact on production application
Fix would cause regression	DO NOT FIX	Would make security worse

1.5.4 5.4 Current Status

Transitive Vulnerabilities in QuizTimer4Zoom:

Package	Version	Severity	Location	Impact	Status
path-to-regexp	6.1.0	HIGH	@vercel/node	Build-time routing	ACCEPTED
esbuild	<=0.24.2	MODERATE	@vercel/gatsby-plugin	Build-time dev server	ACCEPTED
undici	<=5.28.5	MODERATE	@vercel/node	Build-time HTTP client	ACCEPTED

Rationale: - All 3 are in Vercel deployment infrastructure - Not used by application runtime code - Fixing would require downgrading vercel@48.6.0 to 41.0.2 - Downgrade would introduce 13+ new vulnerabilities - Risk acceptance is the responsible decision

1.6 6. Continuous Monitoring

1.6.1 6.1 Ongoing Vulnerability Scanning

Weekly Schedule:

Monday: Automated Scan Day - `npm audit` - Dependency scan - ESLint security scan - Semgrep SAST scan - Report generation

Wednesday: Advisory Check - Review GitHub Security Advisories - Check NVD for new CVEs - Review Zoom Security Bulletins - Assess impact on QuizTimer4Zoom

Friday: Metrics Review - Analyze vulnerability trends - Review remediation status - Update vulnerability dashboard - Team briefing on findings

1.6.2 6.2 Vulnerability Database

Tracked Information: - Vulnerability ID (CVE/GHSA) - Affected package and version range - Discovery date - Severity (CVSS score) - Exploitation status (theoretical/active) - Fix status (available/pending/accepted) - Remediation date - Verification date

Accessibility: - Security Lead: Full access - Development Team: Read-only access - Management: Summary reports

1.6.3 6.3 Metrics & Reporting

Weekly Report Includes: - New vulnerabilities discovered - Vulnerabilities fixed - Average time to remediation - Vulnerability trend (increasing/stable/decreasing) - Compliance status

Monthly Report Includes: - Vulnerability summary statistics - Top vulnerability types - Remediation efficiency metrics - Risk posture assessment - Recommendations for improvement

---

1.7 7. Incident Response for Vulnerabilities

1.7.1 7.1 Critical Vulnerability Incident

When a Critical Vulnerability is Discovered:

1. **Immediate Actions (within 1 hour)**
   - Alert Security Lead
   - Assemble incident response team
   - Assess impact on production
   - Determine if users are at risk
2. **Assessment (within 2 hours)**
   - Verify vulnerability in our version
   - Determine scope of impact
   - Check for active exploitation

- Assess business impact

3. **Remediation (within 24 hours)**
   - Develop and test fix
   - Deploy to production if safe
   - Verify vulnerability is fixed
   - Monitor for issues
4. **Communication (parallel to above)**
   - Notify Zoom of incident
   - Provide status updates every 4 hours
   - Notify affected users (if applicable)
   - Document incident
5. **Post-Incident (within 1 week)**
   - Full incident report
   - Root cause analysis
   - Process improvements
   - Team debrief

1.7.2 7.2 Escalation Procedures

Issue	Time	Escalate To	Action
No response to critical issue	30 min	CTO/Management	Activate backup team
Fix cannot be deployed safely	2 hours	Security Committee	Risk acceptance decision
External exploit confirmed	Immediately	Zoom	Notify immediately
User data at risk	Immediately	Legal/PR	Breach notification prep

---

1.8 8. Vulnerability Management Training

1.8.1 8.1 Required Training

All Development Team Members: - OWASP Top 10 (annual) - Secure Coding Practices (annual) - Dependency Security (quarterly) - Incident Response (annual)

Security Lead: - Advanced Vulnerability Management (annual) - Threat Modeling (annual) - Security Architecture (annual)

1.8.2 8.2 Awareness Program

- Monthly security newsletters
- Security bulletins for critical issues
- Team meetings discussing lessons learned
- Case studies of past vulnerabilities

1.9 9. Integration with Development Lifecycle

1.9.1 9.1 Development Phase

Before Coding: - Review dependency security status - Plan security testing approach - Identify potential vulnerability areas

During Coding: - Security-focused code review - Run linters and SAST tools - Follow secure coding guidelines

Before Pull Request: - All scans must pass - Security issues must be addressed - Code review approval required

1.9.2 9.2 Testing Phase

Test Planning: - Include security test cases - Plan for vulnerability testing - Identify attack vectors to test

Test Execution: - Security testing in test environment - Validation of security controls - Vulnerability scanning

Before Release: - Final security scan - All critical issues resolved - Security approval required

1.9.3 9.3 Deployment Phase

Pre-Deployment: - Final vulnerability scan - Dependency verification - Security checklist review

Post-Deployment: - Production vulnerability scan - Log monitoring for suspicious activity - User impact assessment

---

1.10 10. Third-Party & Open Source Management

1.10.1 10.1 Dependency Onboarding

When Adding New Dependency:

1. **Selection Phase**
   - Evaluate alternatives
   - Check security history
   - Review maintenance status
   - Check community reputation
2. **Assessment Phase**
   - Run `npm audit` with new package
   - Check for existing vulnerabilities
   - Review package.json for issues
   - Check for deprecated warnings
3. **Approval Phase**

- Document justification
- Security Lead approval required
- Alternative analysis saved
- Added to approved list

4. **Monitoring Phase**
   - Regular vulnerability scans
   - Monitor for maintenance
   - Watch for deprecation notices
   - Plan replacement if necessary

1.10.2 10.2 Dependency Version Management

Policy: - Use specific versions for critical packages - Use ^ (caret) for stable APIs - Use ~ (tilde) only for patch updates - Never use * (latest) - Regularly review and update

Current Strategy: - Critical packages: Specific versions - Framework packages: ^ with testing - Utility packages: ^ with regular updates - Development packages: ^ optional

---

1.11 11. Compliance & Standards

1.11.1 11.1 Standards Followed

- **NIST Cybersecurity Framework:** Identify, Protect, Detect, Respond, Recover
- **OWASP:** Top 10, secure coding practices, dependency management
- **CWE:** Common Weakness Enumeration classification
- **CVSS:** Vulnerability severity scoring
- **ISO/IEC 27001:** Information security management

1.11.2 11.2 Regulatory Requirements

- **GDPR:** Breach notification and security measures
- **CCPA:** Vulnerability management for user data protection
- **Zoom Marketplace:** Security requirements and ongoing monitoring

---

1.12 12. Appendix: Vulnerability Management Checklist

1.12.1 Weekly Security Checks

- [ ] Run `npm audit`
- [ ] Review new security advisories
- [ ] Check for critical vulnerabilities
- [ ] Document findings
- [ ] Create tickets for any issues
- [ ] Assign to development team

1.12.2 Monthly Review

- [ ] Review vulnerability metrics
- [ ] Assess remediation progress
- [ ] Update policies if needed
- [ ] Team training/awareness
- [ ] Report to management

1.12.3 Quarterly Assessment

- [ ] Full vulnerability audit
- [ ] Penetration testing (if budget allows)
- [ ] Policy review and updates
- [ ] Process improvement discussion
- [ ] Stakeholder briefing

1.12.4 Annual Review

- [ ] Comprehensive security assessment
- [ ] Policy effectiveness evaluation
- [ ] Industry standards updates
- [ ] Process improvements implementation
- [ ] Training program review

1.13 13. Contacts & Escalation

Primary Security Contact: security@fuzzy.monster **Response Time:** 1 hour for critical issues

For Questions: - Email: security@fuzzy.monster - Website: https://fuzzy.monster

---

Document Status: FINAL ✅ **Effective Date:** October 28, 2025 **Next Review:** October 28, 2026

Fuzzy Monster - QuizTimer4Zoom Vulnerability Management Procedures